Tori Baral
Student number: 22504174

Section 4

1. Why can computers not represent all real numbers exactly?
This is because real numbers are uncountably infinite. Computers, on the other hand, work with finite memory. Any digital representation of real numbers is necessarily approximative.

2. What is the role of the mantissa in a floating-point number?
It represents the significand i.e, the numbers after the decimal point, the fractional part. In the context of digital representations of numbers, the more bits available the mantissa, the higher the precision of the representation.

3. What happens when a number is too large to be represented?
We run into the problem of numeric overflow. when a number becomes too large to be represented within the available exponent range, the result is typically set to infinity. This can alter the behaviour of a computation, often without any explicit warning.

4. Why is float16 more unstable than float32?
It leads to even coarser approximations because it has even less bits to attribute to the mantissa or exponent. This is more unstable because then calculations made with these numbers are even more approximative and imprecise.

5. Why do machine learning models often not use float64?
Because these require large amounts of memory. ML models need to consider memory usage for both the training as well as the inference stages. In the training stage, most ML models use 32 bit and then down to 16 bit for the inference stage.

6. Why does reducing numerical precision (e.g. from float32 to float16) improve both speed and memory usage, and why can this become problematic for training?
Float16 uses less bits and therefore less memory than float32. This can be problematic for training because for example storing parameters in float16 vs float32 can dramatically change the results of the model. float32 has long been the default choice because it offers a good compromise between precision and efficiency

7. Explain why bfloat16 is generally more numerically stable than float16, even though both use 16 bits.
bfloat16 (Brain float 16) is a numerical data type that has the advantage of retaining

the same exponent size as float32 while reducing the mantissa.
Even though they both use 16 bits, bfloat 16 can represent numbers over a wide dynamic range, similar to float32, but with lower precision.

8. In a Hugging Face workflow, why is it often safe to use lower precision for inference but more risky during training?
Training is arguably a more sensitive stage in the ML model, training a model on imprecise numbers could lead to drastically different stages during the inference stage. It is less important during inference. Most computations are done in low precision (typically float16 or bfloat16) while maintaining certain critical variables, such as model weights or accumulated gradients, in higher precision.

Section 5

9. Explain why the two expressions give different results, even though addition is associative over real numbers.
This demonstrates how precision can differ between operations. The first print statement, the two numbers in the inner bracket cancel each other out and finally add c, so we get 1.0.
In the second print statement, however, the 1 is too small to matter and is therefore not considered. This is because float32 uses 23 bits for the mantissa, giving roughly 7 decimal digits of precision. 1.0 is **below the representable precision** at the scale of 1e8 and gets rounded away to zero. So the final calculation is just 1e8 - 1e8 + 0 and it gives 0.0

10. Why is the comparison false in the following code?
As we can see when we print them out, 0.30000000000000004 and 0.3 are not identical.
As we have seen before, the storage of float numbers is never exactly precise due to memory constraints. Adding them makes the situation ever more unstable. 0.1 + 0.2 compounds the rounding errors of two already-imprecise values.

11. Run the following code
What happens here? Identify overflow and underflow.
x: overflow
I also get the following error:

```
/tmp/ipykernel_13782/2854407743.py:3: RuntimeWarning: overflow encountered
in exp
  x = np.exp(1000)
```

This confirms that it is indeed overflow. Overflow occurs when a number becomes too large to be represented within the available exponent range, the result is typically set to infinity.

y:underflow

Underflow is the opposite, it occurs when a number becomes too small in magnitude and is rounded to zero.

12. Why is the following code numerically unstable?
The square root of 1e8 + 1 is really close to the square root of 1e8 (now that we're using float64), so we would assume that subtracting them would result in nearly zero. But the problem comes when we try to subtract them, the small difference in precision is wiped away in the rounding error.
This is called catastrophic cancellation, the leading digits wipe each other out, and we lose most of our significant digits this way.

Section 6

13. What goes wrong?
np.exp(1000) is very large, much larger than what float64 can store. So it overflows to infinity. So calculation inf/inf results in nan, as seen in the warning:

```
RuntimeWarning: invalid value encountered in divide
  return np.exp(x) / np.sum(np.exp(x))
```

14. Compute softmax by hand for a small vector

my vector = [1,2,3]

e ^ 1 = **2.71828182846**

e ^ 2 = **7.38905609893**

e ^ 3 = **20.0855369232**

e^1 + e^2 + e^3 = **30.1928748506**

After applying softmax:

[**0.09003057317**, **0.24472847105** , **0.66524095577**]

15. Compare naive vs stable vs PyTorch
Naive implementation clearly breaks in cases of numeric overflow. The stable version fixes this by subtracting the max first, keeping the exponents manageable. PyTorch does this but under the hood. It also handles edge cases like underflow and is used in the real world most often.

16. Why does subtracting the maximum fix the problem?

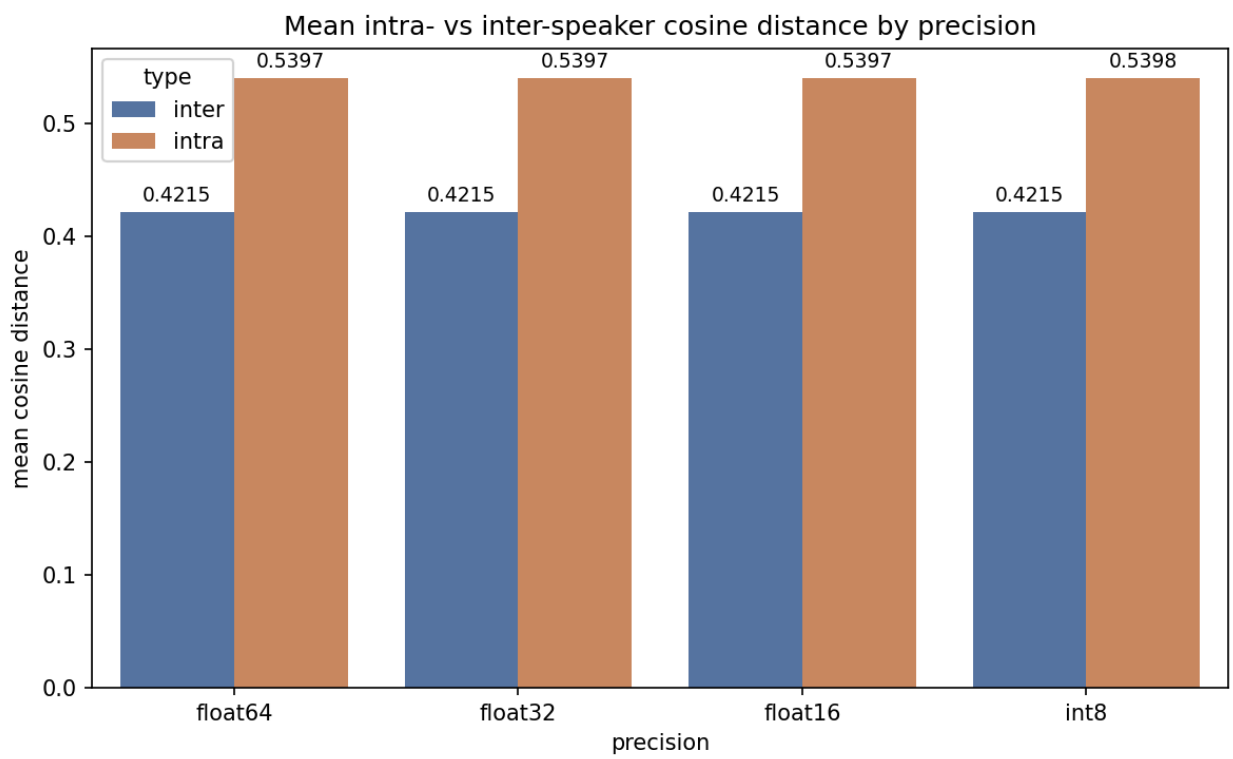
Because the biggest input becomes 0 after subtracting the max, so the biggest exponent you ever compute is $e^0 = 1$. Everything else is e to a negative number, which is always between 0 and 1. Like this, overflow doesn't happen anymore.

Section 7

Report:

I extracted wav2vec 2.0 (facebook/wav2vec2-base) representations for 239 recordings from 19 speakers across 13 words of the Russian-French interference corpus found on https://www.ortolang.fr/market/corpora/ru-fr_interference/v1.

I mean-pooled frame-level hidden states from the last transformer layer to produce one 768-dimensional vector per recording, stored in float32. Mean-pooling was chosen as an effective way to collapse variable-length sequences into fixed-size representations suitable for distance computation. Cosine distances were then recomputed after casting to float64, float32, float16, and 8-bit quantisation. This showed a direct comparison of how precision affects the geometry of the representation space.



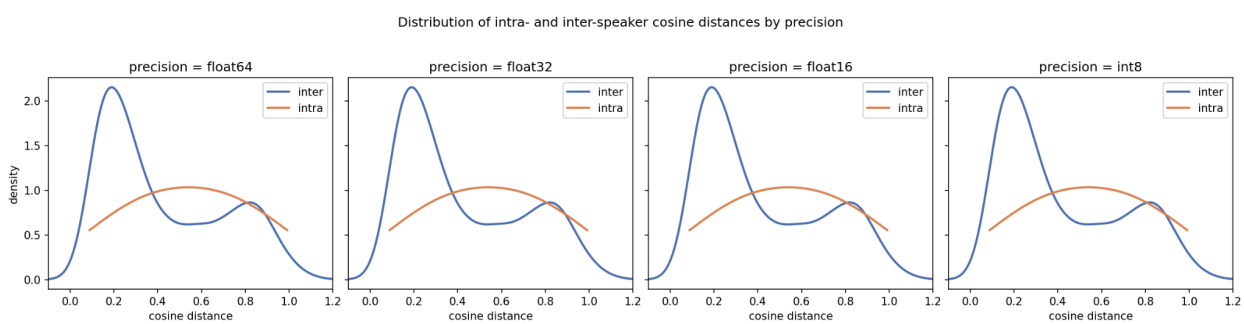
The 8-bit scheme I chose used min-max linear quantisation per-vector. Each 768-dim vector is scaled to [0, 255], shifted to [-128, 127], and stored as int8. A global scale would be skewed by extreme values across the dataset, so scaling per-vector ensures that the 256 quantisation levels actually cover a meaningful and representative range for each individual embedding. To

compute distances, vectors are ‘dequantised’ back to float32. However, the precision loss is already present at this point and does not recover with ‘dequantisation’. i.e, the information lost during the int8 conversion is gone permanently.

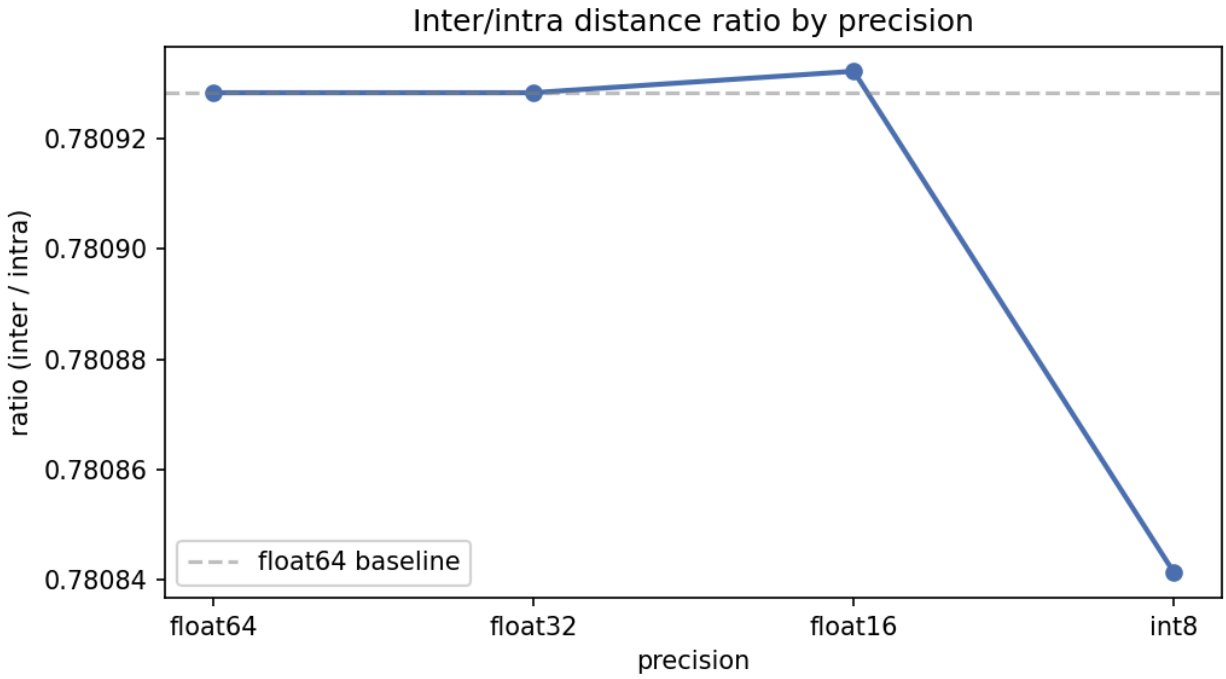
Cosine distances were computed for all pairs sharing the same word, labelled intra-speaker (same speaker, different repetition) or inter-speaker (different speaker, same word). Inter-speaker distances (~0.42) were consistently lower than intra-speaker distances (~0.54) across all precision levels. This is an odd result, it suggests that the wav2vec 2.0 representations cluster more strongly by the lexical content of the phrase than by speaker identity. In other words, two different words produced by the same speaker are farther apart in embedding space than the same word produced by two different speakers. This suggests the model learned strong lexical structure, rather than encoding speaker-specific features.

The results I got are shown here:

type	inter	intra	ratio
float16	0.421471	0.539703	0.780932
float32	0.421472	0.539706	0.780928
float64	0.421472	0.539706	0.780928
int8	0.421491	0.539791	0.780841



Across all four precision levels, mean distances are stable to four decimal places. Float32 and float16 are almost identical to float64, confirming that the extra precision of float64 adds essentially nothing for this type of task. Int8 introduces a slightly larger deviation, but the relative ordering of inter vs intra distances is fully preserved. The inter/intra ratio varies by less than 0.0001 across all precisions (float64: 0.7809, int8: 0.7808), confirming that reducing precision does not meaningfully change the geometry of the embedding space.



The graph above says that inter/intra ratio varies by less than 0.0001 across all precisions (float64: 0.7809, int8: 0.7808). This confirms that reducing precision does not dramatically alter the structure of the representation space.

However in terms of storage and time taken, there is a much larger difference when reducing precision:

Format	Storage (239 vectors)	Distance matrix time
float64	1.47 MB	0.916s
float32	0.73 MB	0.033s
float16	0.37 MB	0.030s
int8	0.73 MB	0.028s

Float64 is approximately 28 times slower than float32 for distance computation, despite offering no measurable improvement in results for this task. Float16 achieves half the storage footprint of float32 with negligible accuracy cost, making it particularly attractive for large-scale deployments where memory is a bottleneck. Int8 matches float16 in computation speed but uses the same storage as float32, since vectors are ‘dequantised’ back to float32 in memory before distance computation.

Conclusion:

Float16 is the most practical format for this type of task. Float64 is unnecessary. The better precision it provides costs nearly 30 times the computation time with no great benefit. Float32 remains a solid default with good speed, but float16 offers an additional 2x reduction in storage with almost little change in precision. The results suggest that the representation space is robust to precision reduction.